

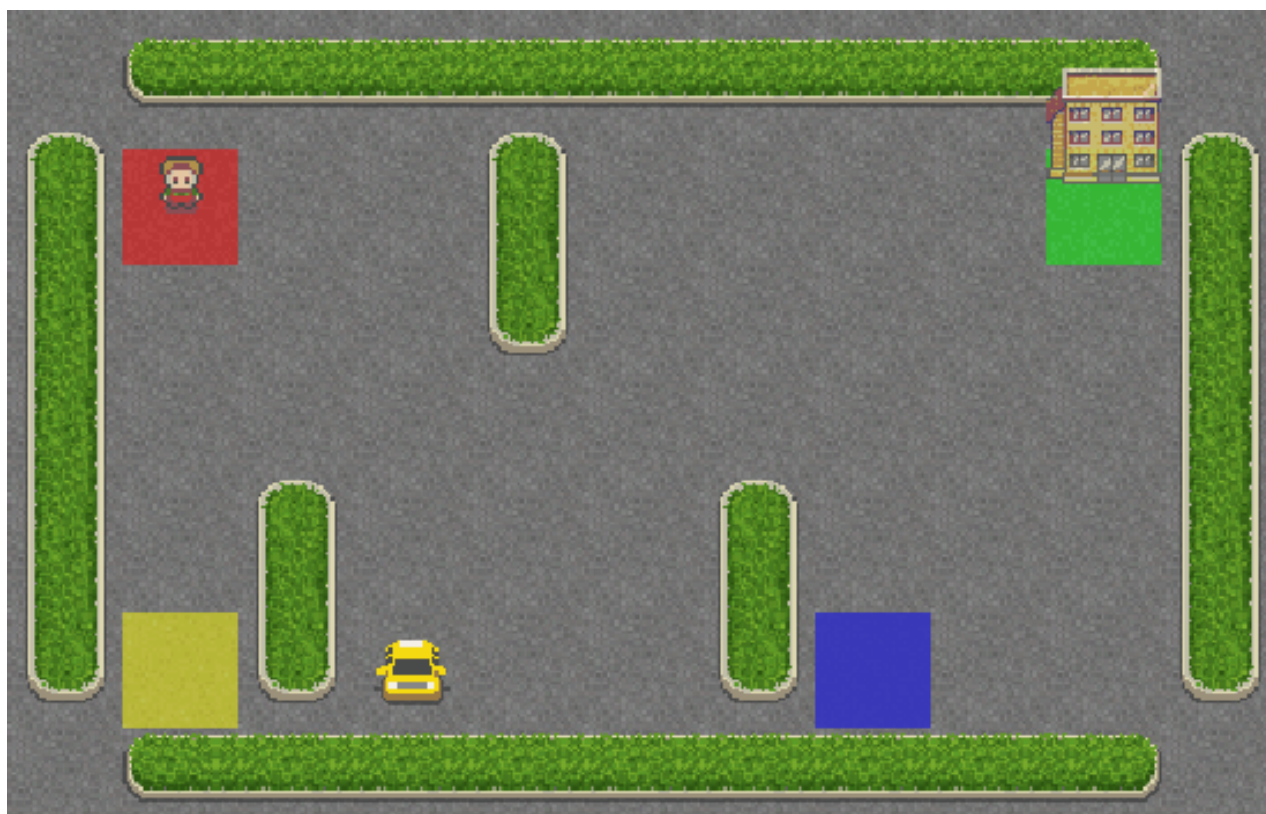
Projeto 2 — “Aprendendo a dirigir com Q-Learning”

Agora que você já tem familiaridade com alguns ambientes clássicos do Gymnasium — como o Frozen Lake, em que usamos Q-Learning, e o Blackjack, em que exploramos políticas fixas —, é hora de darmos mais um passo adiante. Neste projeto, vamos colocar o agente em uma situação um pouco mais próxima da realidade: dirigir um táxi em uma cidade com posições bem definidas.

Neste ambiente, o táxi deve buscar um passageiro em um ponto de embarque e levá-lo até o destino correto. Porém, o agente não sabe de antemão quais são as melhores sequências de ações. Ele terá que aprender a navegar nesse mundo através da experiência, escolher quando pegar e quando deixar o passageiro, e a planejar rotas que maximizem sua pontuação.

O ambiente

A ideia é que você use o ambiente Taxi-v3 da biblioteca Gymnasium. Ele é composto por uma pequena cidade representada por uma grade 5x5. Nela, um táxi precisa se locomover até a posição de um passageiro, pegá-lo e levá-lo até um destino determinado, como ilustra a figura a seguir. Existem quatro pontos fixos no mapa: vermelho (R), verde (G), amarelo (Y) e azul (B). Esses pontos funcionam como possíveis locais de embarque e desembarque. A cada episódio, a posição inicial do táxi, a posição do passageiro e o destino são sorteados aleatoriamente.



Ações possíveis do táxi:

- 0: mover para o sul
- 1: mover para o norte
- 2: mover para o leste
- 3: mover para o oeste
- 4: pegar passageiro
- 5: largar passageiro

Estados representam combinações entre:

- Posição atual do táxi (25 possíveis, pois o mapa é uma grade 5×5, como ilustrado abaixo)

None

```
+-----+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+-----+
```

- Posição do passageiro (4 locais fixos + “dentro do táxi”)
- Destino do passageiro (4 possibilidades)

No total, existem 500 estados discretos possíveis ($25 \times 5 \times 4$), embora nem todos sejam alcançáveis durante um episódio. Por exemplo, situações onde o passageiro já está no destino não ocorrem no início do jogo. O Gymnasium codifica cada estado como um número inteiro entre 0 e 499.

Recompensas:

- +20 ao deixar o passageiro no destino certo
- -1 por cada passo dado (custo do tempo gasto)
- -10 se tentar pegar ou deixar em lugar inválido

O episódio termina quando o passageiro chega ao destino ou quando o limite de passos (200, por padrão) é atingido.

As máscaras de ações

Diferente dos ambientes anteriores, no Taxi-v3 existe uma ferramenta extra chamada máscara de ações, que serve para indicar quais ações fazem sentido no estado atual.

Antes de falarmos da máscara em si, é importante relembrar que o ambiente retorna duas estruturas diferentes a cada passo (episódio):

- *observation* (obs): representa formalmente o estado do ambiente. No Taxi, esse valor é um número inteiro entre 0 e 499, que codifica a posição do táxi, a posição do passageiro e o destino escolhido. Essa codificação é feita pelo Gymnasium através da fórmula: $estado = ((linha_{taxi} \cdot 5 + coluna_{taxi}) \cdot 5 + local_{passageiro}) \cdot 4 + destino$. É esse número que usamos para indexar as linhas da Q-Table.
- *info*: é um dicionário com informações adicionais, que não fazem parte da observação em si, mas ajudam a controlar o comportamento do agente ou a otimizar o treinamento. É dentro desse dicionário que está a *action_mask*.

Essas duas estruturas têm funções diferentes e complementares: *observation* diz “onde estamos e qual é o estado atual”, enquanto *info* diz “quais ações fazem sentido no estado atual”.

Podemos pensar da seguinte forma: imagine que o táxi está encostado em uma parede à esquerda. Mandar ele “andar para a esquerda” não muda nada. Ele bate na parede e fica parado, perdendo tempo e recebendo -1 de recompensa. Ou ainda: se você mandar o táxi pegar um passageiro quando não tem ninguém no quadrado em questão, a ação também não tem efeito e o agente recebe -10 de penalidade.

Para evitar esse tipo de ação improdutiva, o ambiente entrega junto com cada observação um vetor de 0 e 1, chamado *action_mask*. Esse vetor nos mostra quais ações são válidas naquele momento:

Python

```
info["action_mask"] = [1, 1, 1, 0, 0, 1]
```

Nesse caso, as ações 0, 1, 2 e 5 estão disponíveis (1 = permitido), e as ações 3 e 4 estão bloqueadas (0 = inválidas). Ou seja, o agente pode se mover para o sul, norte e leste, além de largar o passageiro, mas não deveria se mover para oeste ou embarcar um passageiro.

Durante o treinamento com Q-Learning, usamos essa máscara para guiar as decisões do agente:

- Ações guiadas pelo caso ϵ : sorteie somente entre as ações válidas

Python

```
action = env.action_space.sample(info["action_mask"])
```

- Ações guiadas pelo caso $(1-\epsilon)$: escolha a melhor ação (argmax) apenas entre as que têm 1 na máscara, ignorando as inválidas.
- Na atualização da Q-Table, ao calcular o valor máximo no próximo estado ($\max Q(s', a')$), também considere apenas as ações válidas em s' .

Nos ambientes anteriores (Frozen Lake e Blackjack), podíamos ignorar o campo *info* sem problemas. No Taxi-v3, ignorar a máscara faz com que o agente passe boa parte do tempo tentando realizar ações que não mudam nada ou geram penalidades, atrasando consideravelmente o aprendizado.

O problema

No ambiente Taxi, além de navegar no mapa evitando paredes, como no Frozen Lake, o agente precisa entender quando deve realizar ações especiais (embarcar e desembarcar o passageiro) no momento e lugar corretos. Isso torna a busca pela política ótima um pouco mais exigente que nos ambientes anteriores.

Nosso objetivo será treinar um agente usando Q-Learning para resolver o Taxi-v3. O agente irá atualizar a Q-Table a cada experiência, refinando suas decisões com base nas recompensas recebidas. Nela, cada linha representa um estado e cada coluna representa uma ação. A cada passo, o agente:

1. Observa o estado atual
2. Escolhe uma ação (seguindo a política ϵ -greedy, usando máscara)
3. Recebe uma recompensa e observa o próximo estado
4. Atualiza a Q-Table usando a equação vista nos laboratórios anteriores¹

Ao longo dos episódios, é esperado que os valores da Q-Table converjam, indicando para cada estado qual é a melhor ação a ser tomada.

As Tarefas

1. Exploração do ambiente

- Tente fazer o mesmo processo dos laboratórios anteriores: inicialize o Taxi-v3, execute alguns episódios com ações aleatórias, SEM usar a máscara, e observe como o ambiente se comporta.
- Tente fazer o mesmo processo dos laboratórios anteriores: inicialize o Taxi-v3, execute alguns episódios com ações aleatórias, já usando a máscara, e observe como o ambiente se comporta.
- Compare as duas soluções (com e sem máscara) e identifique qual delas parece mais promissora? Alguma das soluções apresenta alguma regularidade ou progresso no aprendizado?

2. Implementação do Q-Learning

- Crie a Q-Table com dimensões (num_estados, num_acoes) e implemente a regra de atualização.
- Use uma política ϵ -greedy com máscara para guiar as ações. Se necessário, relembre seu funcionamento nas soluções dos labs 5 e 6².

¹ https://docs.google.com/document/d/1gbc8Lzp9zM2g3ofBKzQE_TOmDh7FvLELEIwQljR2i54/edit?usp=sharing

² Laboratório 5: <https://drive.google.com/file/d/1XUrRcm4Zh0IMn4aEHg-aFdsJoqHr6Tz/view?usp=sharing>

Laboratório 6: <https://drive.google.com/file/d/1W5K9Td65ZvDW028wOOZrFyjC6q5WUKHU/view?usp=sharing>

3. Treinamento

- Escolha hiperparâmetros (taxa de aprendizado α , fator de desconto γ , número de episódios, ϵ inicial e decaimento) e treine seu agente.
- Acompanhe a evolução da recompensa média ao longo dos episódios através de um gráfico de recompensas em função dos episódios, tal como fizemos até agora.

4. Avaliação da política aprendida

- Teste o agente com $\epsilon = 0$ e avalie sua taxa de sucesso.
- Visualize a política aprendida rodando o ambiente com a parte gráfica ativada.

A apresentação

Você deve apresentar a sua solução no dia 07/11, no LCA ou H.IAAC, a depender da disponibilidade de espaço.

Mais informações

Visão geral do ambiente Taxi na documentação oficial do Gymnasium, incluindo explicações sobre *action masking*:

https://gymnasium.farama.org/environments/toy_text/taxi/

Propostas de solução dos laboratórios anteriores:

 solucoes_gym